

Function ^{Argument} ^{تابع} ^{MDN} ^{Call back} ^{التي اقر في المتغير تابع} ^{Function} ^{Parameter} ^{التي}

Callback is a function passed into another function as Argument, which is then invoked inside the outer function to complete some kind of action

Function one(x) {
 console.log('one'),
 x() } ^{Call Back}

one(two)
one(three) ^{Call Back as one}

Promise

getData1 ((res1)=7)
 ——— 2 ((res2)=7)
 ——— 3 ((res3)=7)
 ——— 4 ((res4)=
 console.log('done'))

فلا في نفس الوقت
كود شكلا متساوي
جدا وعلا لانه كده
Promise
علا

Asynchronous JavaScript and XML.

وهذا يعني أننا نتعامل مع request و response

Client

request

response

Web Server

وأنواع مختلفة من request هي delete, get, post, put, patch, etc.

API: Application Programming Interface

وهو في الحقيقة API هي عبارة عن شكل XML الذي يمكن استخدامه

Call Stack

هو عبارة عن الذاكرة التي تخزن معلومات عن الدالة التي تم استدعاؤها في الوقت الحالي وما يجب فعله بعد انتهاء الدالة من التنفيذ

عند استدعاء دالة ما يتم تخزين معلومات عنها في الذاكرة مثل اسم الدالة، مكان استدعاؤها، المتغيرات المحلية، etc.

Call back queue

وهو عبارة عن قائمة انتظار للدوال التي تم استدعاؤها ولكنها لم يتم تنفيذها بعد. عندما يتم تنفيذ الدالة في القائمة، يتم استدعاؤها مرة أخرى.

هناك دالة واحدة تسمى event loop والتي تتحقق باستمرار من قائمة انتظار الدوال. عندما تكون الدالة جاهزة للتنفيذ، يتم استدعاؤها مرة أخرى.

بمجرد انتهاء الدالة من التنفيذ، يتم إزالتها من قائمة انتظار الدوال. إذا كانت الدالة تحتاج إلى استدعاء دالة أخرى، يتم استدعاء الدالة مرة أخرى.

وطبعاً لو موزلناش check ان readyState==4
يطبع response وطبعاً اقدر استعمل لا This فيها

const data = this.responseText

و Song وصلنا منه جوده صوتي و Teet بالاسم ده

V262

V263 Promise

اتنا كان في مشكلة بتحصل هو وان في كل مرة بعد في XML HTTP
يوصلي مشكلة Call back hell لئلا نواجه API ولما وز ارتبهم
ويجولي بالترتيب فلانم اعمد حاجيت و Async, Await في
XML HTTP فيحصل Call back في كل مرة على ال Function او ال invoked في
الصح

Call back

```
function one() {  
  console.log('one')  
}
```

```
function two() {  
  console.log('Two')  
  one()  
}
```

```
function three() {  
  console.log('Three')  
  one()  
  two()  
}
```


Xml HTTP Request

const request = new XMLHttpRequest()

معرفة حالة يكون في الmethod الى بتخيل تمام مع backend
وده الى عميل انا انا نستخدم activeX object سيطرنا بل خالص
اولا انا بتعامل معه انا انا نعمل نسخة ونحتر لا بيها وده متغير
new

وهو كده انا علور action فتحه في method و open
وهو بتاخذ حاجته فتح method, url

request.open("get", url)

الاعتراف في الكايت انا مكتوب وهو default هياندها get
وبعد كده بتعطيهم على السيرفر

request.send

وعشان انت اشوف انا انا بطلب من السيرفر اني يجيب
الردا response

request.response

بس هنا طبعا فنانا مشكله في انا الخ. الخ بتلقها
اني اعلمها انتظرها عشان استاها من call

call, function, addEvent / listener

request.onreadystatechange

فكره انا معايا ال API ب 501

العمل الحقيق هو deep clone وهو Structure Clone

const kernelOS = structureClone (garage16)

ده بيقل نسخة كاملة مستقله بيرون الkernelOS مش بتاين على
النسخه الاصليه فلو حذفته او حذفت من
على الgarage16

205. memory mangment garbage

function
بف يا عمل انا array, object, string على خطواها
1 allocate
2 use
3 release
مسح الذاكرة الى انت مش محتاجها

Release: garbage collector
المسح هنا بيحصل بشكل تلقائي عن طريق
mark and sweep Algorithm

هنا بقى انا المسح يتم في stack:
الاعتراف بيخزن: variable
execution context

execution context
اولا الfunction context
مثال

function calc () {
let x = 10
let y = 20
calc ()
}

بعد ما calc يخلص
بيشالوا من الذاكرة تلقائي بيرون
garbage collector
المشغلات ال memory بس هي الى مشغله

Day 5

section 26

Synchronous code:

بمعناها هو الكود الذي ينتهي هنا by line بالترتيب ويحصل execution ثم end وده جزء من execution context
 بجمع block مثلا على alert وده لو بجمع عمليات زي setTime
 لست مانيه انقطع على alert لازم كل واحد ينتظر الثاني وظيفه بيكون
 انه المصفى تكون ببطيئة ومش كساحرة بيديه Async هو العمليات التي يحصلها
 Async rows: هي هنا في شكل انه هو اول non فني مش هيو قف عملاي
 سايه عشان الترتيب او انج البرمجة هو هنا الى هيو قف عملاي
 هيو قف لكن الترتيب مش مهم عنده عشان هو بيخزن اى
 عملية هتتاخذ وقت في بوكس الذاكرة ودها الى اننا نطنا عملة في
 execution والديجرام الى هيوصلوا انتظار زي مثلا

Async: Execute after task That runs in the background

- 1 non blocking
- 2 execution doesn't wait for async task to finish its work
- 3 call back Function don't mark code async.

نا هيرجع ال Buffer وكدا لك يرضو في detection

console.log (this)

يعني لو كنت لا this في global scope هيرجع window
 في regular function بيختلف بقى على حسب ال strict mode
 لو في strict mode هيرجع كاري على عشان هيرجع undefined
 لو هينس هيرجع ال window عشان ال function مش قوزن لا في object

V102 Regular function Vs Arrow

نفس الكلاس الى فوق بالظبط

V103 memory management

memory management

بمعناها ماعلم بقى
 اننا فاسكريبت بي عمل حاجتين:

1) بخصص مكان في memory لاي value جديد زي x=10
 2) تبيع ال value الى مكانا خاصا لا هيو وده بيحصل او تخلص

garbage collector انت مش بتحدد مكان في الذاكرة باليد
 انواع البيانات: Primitive Vs Object

Primitive Vs Object

Object, Array, Function, map
 ال object, array, function, map
 ال object, array, function, map
 ال object, array, function, map

Stack: ممتلئ بالقيمة
 Primitive Value, Reference Object

Heap: ممتلئ في الذاكرة نفسها

نوع الذاكرة ممتلئ بنسخة Copy من الذاكرة نفسها
 Copy من الذاكرة نفسها
 Copy من الذاكرة نفسها

let location = 'City: Sober'
 let newLocation = location
 الى جها هو الذاكرة نفسها
 الذاكرة نفسها
 الذاكرة نفسها

Primitive Value = Copy The Value
 Address Object = Copy The Reference

Object Reference

ممتلئ بالذاكرة نفسها
 الى الذاكرة نفسها
 الى الذاكرة نفسها
 الى الذاكرة نفسها

const faragElle = {
 first Name: 'FaragElle',
 last Name: 'Aref',
 const kerolos = {
 last Name: 'Abo El Farag',
 kerolos, last Name: 'Abo El Farag',
 Spread Operator

هذا الى الذاكرة نفسها
 الى الذاكرة نفسها
 الى الذاكرة نفسها
 الى الذاكرة نفسها